

AWS ACADEMY PROJECT PROOF

Microservices and CI/CD Pipeline Builder

Megha Saha | USN: 25MCAR0194

Project Implementation Steps

1. Access the Development Environment

Use AWS Cloud9 as the development environment and manage the application source code with Git and AWS CodeCommit.

2. Split the Monolithic Application

Separate the application into two independent services: Customer Microservice and Employee Microservice. The Employee service handles supplier-management functionality.

3. Containerize the Microservices

Create Dockerfiles, build and test the Customer and Employee containers, and expose the application on port 8080.

4. Push Images to Amazon ECR

Create the customer and employee ECR repositories, tag the Docker images with their ECR URIs, and push the images to ECR.

5. Create ECS/Fargate Environment

Create the microservices-serverlesscluster cluster, register the task definitions, and create the Customer and Employee ECS services using Fargate.

6. Configure ALB and Target Groups

Create customer and employee target groups with IP target type, configure the microservicesLB listeners, and route /admin/* requests to the Employee service.

7. Configure CodeDeploy Blue/Green Deployment

Create the microservices-app CodeDeploy application and customer/employee deployment groups using two target groups and traffic control.

8. Create Customer CI/CD Pipeline

Create update-customer-microservice using CodeCommit and ECR sources and Amazon ECS (Blue/Green) deployment through CodeDeploy.

9. Create Employee CI/CD Pipeline

Create update-employee-microservice using the employee ECR image, task definition, AppSpec file, and microservices-employee deployment group.

10. Verify Blue/Green Deployment

Run the pipelines and verify that CodeDeploy successfully creates the replacement task set and shifts traffic.

11. Verify Application and Database

Open the Employee /admin/suppliers page and verify supplier records from the RDS/MySQL database.

12. Scale and Verify Final State

Scale customer-microservice to 3 tasks, keep employee-microservice at 1 task, and verify both services are ACTIVE and running.

Screenshot Evidence

Evidence 1: Course Completion Certificate

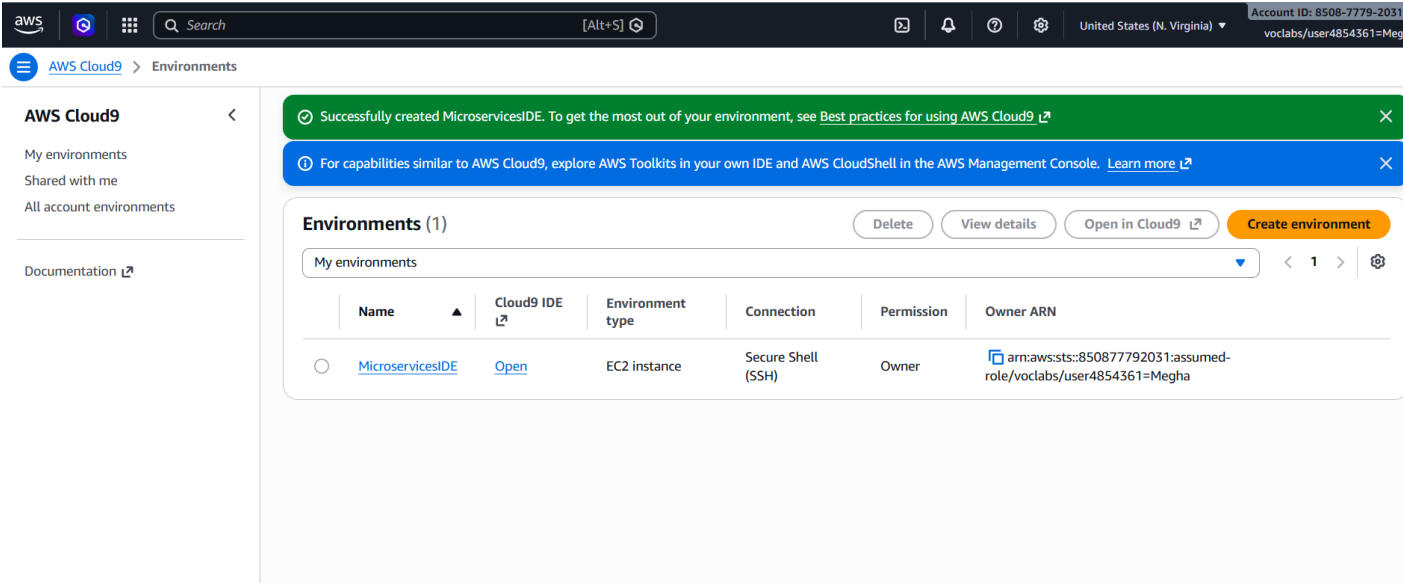
Step: Complete the AWS Academy Microservices and CI/CD Pipeline Builder course/lab activities and retain the course completion certificate as evidence.



Proof: AWS Academy Graduate certificate for the Microservices and CI/CD Pipeline Builder course.

Evidence 2: AWS Cloud9 Development Environment

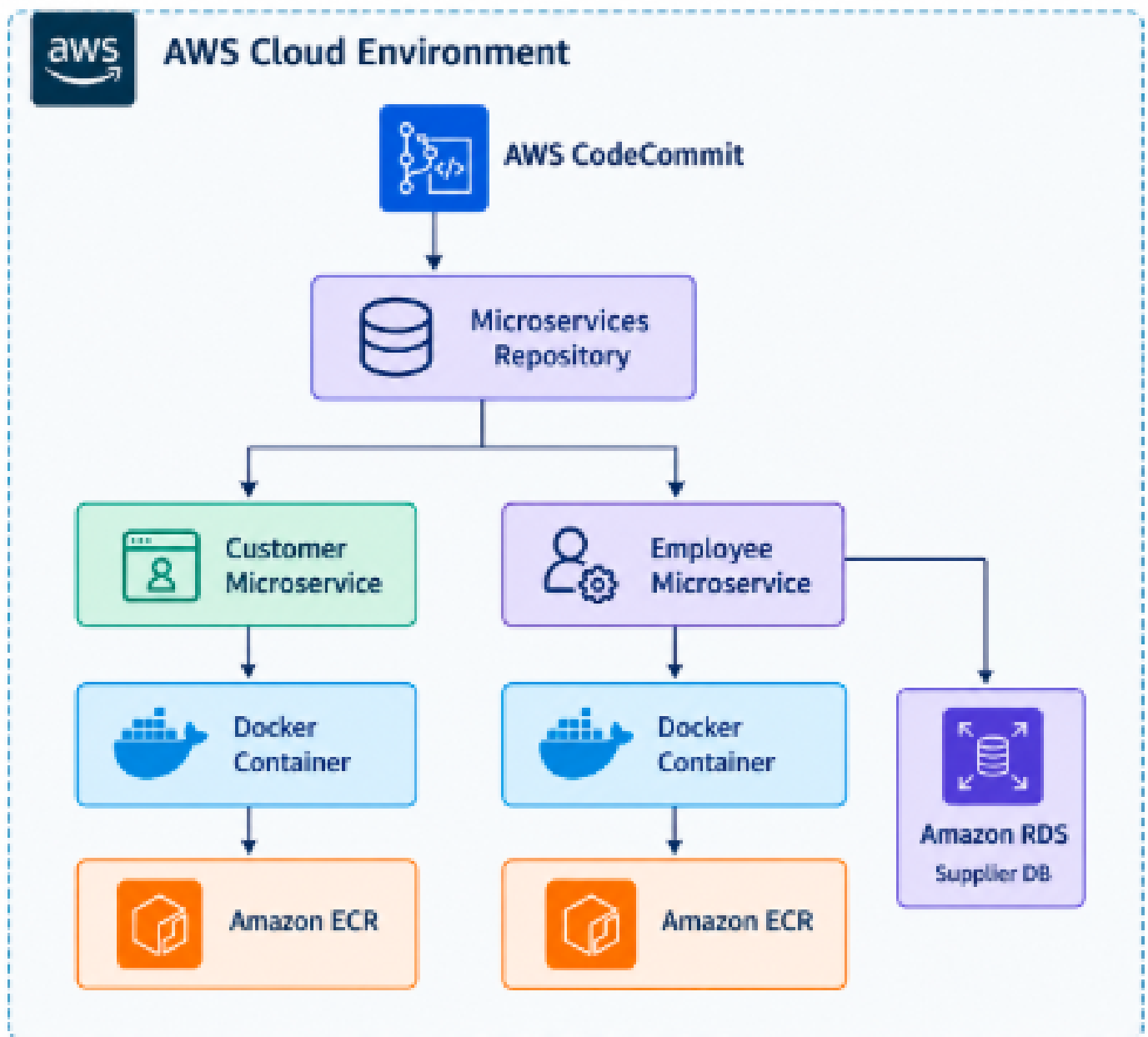
Step 1 – Development Environment: Open AWS Cloud9, create/open the MicroservicesIDE environment, and use the Cloud9 terminal for the application and deployment work.



Proof: MicroservicesIDE Cloud9 environment used during the project.

Evidence 3: Project Architecture

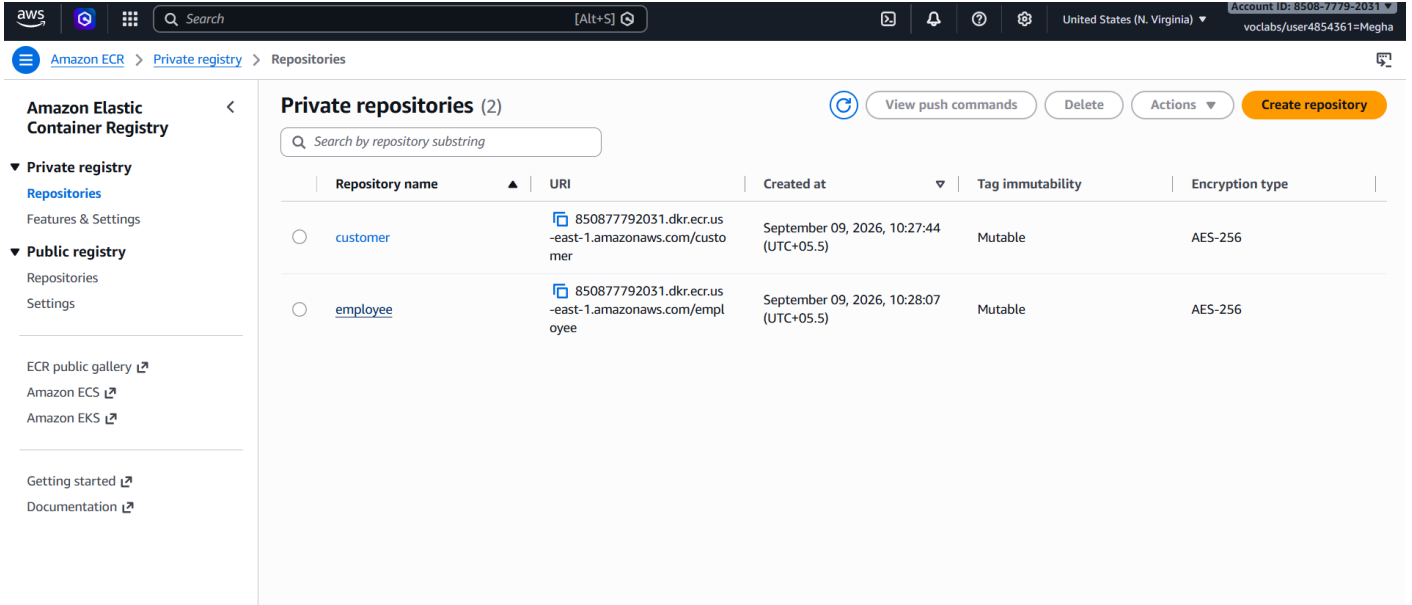
Step 2 – Microservices Design: Separate the monolithic application into Customer and Employee microservices and define the AWS components used for source control, containers, deployment, networking, and database access.



Proof: Architecture diagram showing the implemented microservices workflow.

Evidence 4: Amazon ECR Repositories

Step 3 – Containerization and Registry: Build Docker images for the Customer and Employee services, create separate Amazon ECR repositories, tag the images with their ECR repository URIs, and push the images.



The screenshot displays the Amazon Elastic Container Registry (ECR) console. The left sidebar shows the navigation menu with 'Private registry' selected. The main content area is titled 'Private repositories (2)' and contains a table listing the repositories. The table has columns for Repository name, URI, Created at, Tag immutability, and Encryption type. Two repositories are listed: 'customer' and 'employee'. Both were created on September 09, 2026, at 10:27:44 (UTC+05.5) and 10:28:07 (UTC+05.5) respectively. Both have a 'Mutable' tag immutability and 'AES-256' encryption type.

Repository name	URI	Created at	Tag immutability	Encryption type
customer	850877792031.dkr.ecr.us-east-1.amazonaws.com/customer	September 09, 2026, 10:27:44 (UTC+05.5)	Mutable	AES-256
employee	850877792031.dkr.ecr.us-east-1.amazonaws.com/employee	September 09, 2026, 10:28:07 (UTC+05.5)	Mutable	AES-256

Proof: ECR contains the Customer and Employee repositories.

Evidence 5: Amazon ECS / Fargate Deployment

Step 4 – ECS Environment: Create the Fargate-compatible ECS cluster, register the Customer and Employee task definitions, create the ECS services, and connect them to the Application Load Balancer target groups.

The screenshot displays the Amazon Elastic Container Service (ECS) console interface. The top navigation bar shows the AWS logo, a search bar, and the account ID: 8508-7779-2031. The breadcrumb trail indicates the path: Amazon Elastic Container Service > Clusters > microservices-serverlesscluster > Services.

The main content area is titled 'microservices-serverlesscluster' and shows the cluster's status as 'Active'. The 'Cluster overview' section provides key metrics: Cluster ARN (arn:aws:ecs:us-east-1:85087779:...), Status (Active), CloudWatch monitoring (Default), Action logs (Turned off), Services (2 active | 0 draining), Tasks (4 running | 0 pending), and Registered container instances (-).

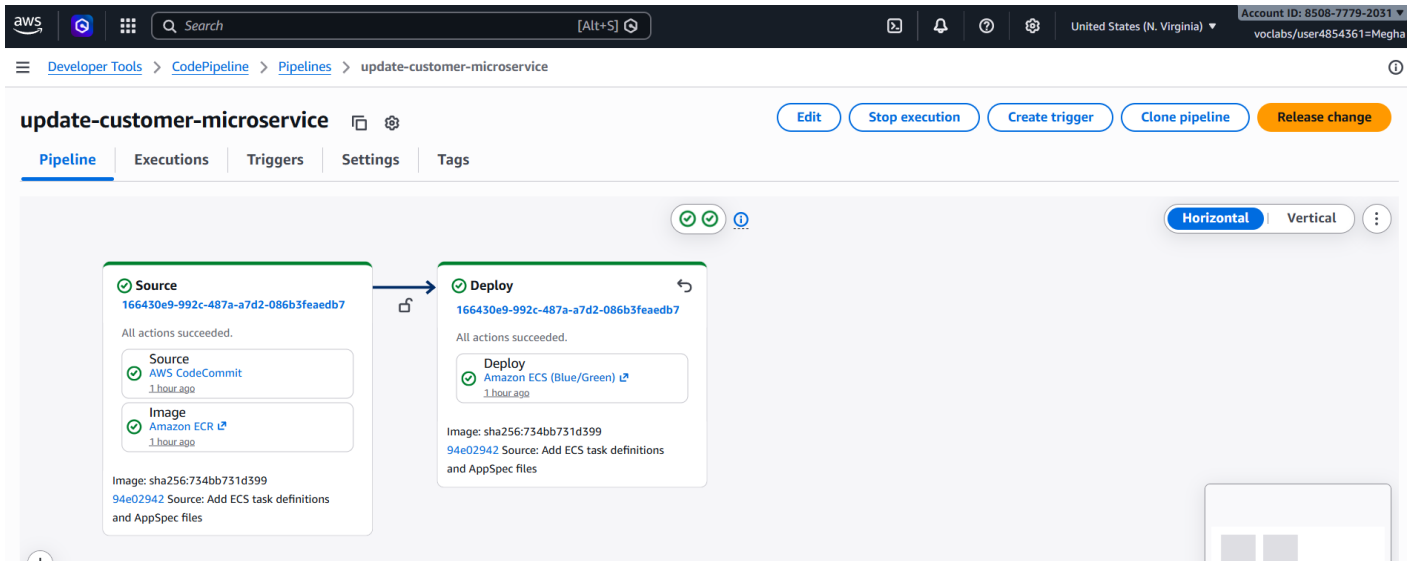
The 'Services' tab is selected, showing a list of two services: 'customer-microservice' and 'employee-microservice'. Both services are in an 'Active' state. The 'customer-microservice' has 3/3 tasks running, and the 'employee-microservice' has 1/1 task running. The table columns include Service name, ARN, Status, Scheduling strategy, Launch type, Task definition, and Deployments and tasks.

Service name	ARN	Status	Scheduling strategy	Launch type	Task definition	Deployments and tasks
customer-microservice	arn:aws:ecs:us-east-1:85087779:...:customer-microservice	Active	Replica	Fargate	customer-microservice	3/3 Tasks running
employee-microservice	arn:aws:ecs:us-east-1:85087779:...:employee-microservice	Active	Replica	Fargate	employee-microservice	1/1 Tasks running

Proof: Final ECS cluster shows both services active with the required running tasks.

Evidence 6: Customer CI/CD Pipeline

Step 5 – Customer Pipeline: Create the update-customer-microservice pipeline using CodeCommit for deployment files, Amazon ECR as the image source, and Amazon ECS (Blue/Green) with CodeDeploy for deployment.



Proof: Customer Source and Deploy actions completed successfully.

Evidence 7: AWS CodeDeploy Blue/Green Deployment

Step 6 – Blue/Green Deployment: Configure the CodeDeploy application and customer/employee deployment groups with the ALB listeners and two target groups, then run the deployments through CodePipeline.

The screenshot shows the AWS CodeDeploy console for the application 'microservices-app'. The left sidebar lists navigation options under 'Developer Tools' and 'CodeDeploy', including 'Source', 'Artifacts', 'Build', 'Deploy', and 'Pipeline'. The main content area shows 'Application details' for 'microservices-app' on the 'Amazon ECS' compute platform. Below this, the 'Deployment groups' tab is active, displaying a table with two groups: 'microservices-customer' and 'microservices-employee', both with a 'Succeeded' status. The table also shows the last attempted and successful deployment times and trigger counts.

Name	Status	Last attempted deploym...	Last successful deployment	Trigger count
microservices-customer	✔ Succeeded	Sep 9, 2026 8:00 PM (UTC...	Sep 9, 2026 8:00 PM (UTC...	0
microservices-employee	✔ Succeeded	Sep 9, 2026 9:01 PM (UTC...	Sep 9, 2026 9:01 PM (UTC...	0

Proof: microservices-app shows successful customer and employee deployment groups.

Evidence 8: Employee Microservice Output

Step 7 – Employee Service Verification: Deploy the Employee Microservice and access the /admin/suppliers route to verify supplier-management functionality and database integration.

← → ↻ ⚠ Not secure 13.220.240.229:8081/admin/suppliers ☆ M ⋮



Manage coffee suppliers

Administrator home
Customer home

All suppliers

Name	Address	City	State	Email	Phone	
Megha	Btm	Bangalore	Karnataka	megha.saha2409@gmail.com	4155551212	edit
Sabeena	Whitefield	Los Angeles	US	nwolf@example.com	4155551214	edit

[Add a new supplier](#)

Proof: Employee supplier-management page displays supplier records.

Evidence 9: Supplier Database – RDS / MySQL

Step 8 – Database Verification: Connect to the supplier database, select the COFFEE database, display the suppliers table, and verify that supplier records are available to the Employee Microservice.

```
| mysql |
| performance_schema |
| sys |
+-----+
6 rows in set (0.03 sec)

mysql> USE COFFEE;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_COFFEE |
+-----+
| suppliers |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM suppliers;
+----+-----+-----+-----+-----+-----+
| id | name   | address   | city       | state   | email |
+----+-----+-----+-----+-----+-----+
| 1  | Megha  | Btm       | Bangalore  | Karnataka | megha.saha2409@g |
| 2  | Sabeena | Whitefield | Los Angeles | US       | nwolf@example.co |
+----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> exit
Bye
ubuntu@ip-10-16-10-11:~/resources/codebase partner$
```

Proof: suppliers table and records are displayed.

Evidence 10: Final ECS Verification

Step 9 – Scaling and Final Verification: Scale customer-microservice to 3 desired tasks, keep employee-microservice at 1 task, wait for ECS stabilization, and verify the final service status.

```
voclabs:~/environment $ aws ecs describe-services \
  --cluster microservices-serverlesscluster \
  --services customer-microservice employee-microservice \
  --query 'services[*].[serviceName,desiredCount,runningCount,status]' \
  --output table \
  --no-cli-pager
```

DescribeServices			
customer-microservice	3	3	ACTIVE
employee-microservice	1	1	ACTIVE

```
voclabs:~/environment $
```

Proof: Terminal verification shows customer-microservice 3/3 and employee-microservice 1/1, both ACTIVE.

Final Result

- Customer and Employee microservices were separated and containerized using Docker.
- Images were stored in Amazon ECR and deployed on Amazon ECS/Fargate.
- ALB target groups and listener rules were configured for application routing.
- CodePipeline and CodeDeploy completed the blue/green CI/CD deployment workflow.
- Employee supplier management was verified using the database-backed /admin/suppliers route.
- Final ECS verification showed 3 running Customer tasks and 1 running Employee task.